CCA 3: COMPUTER VISION ASSIGNMENT

Practical Implementation & Project Documentation

Student Information

Field	Details
Student Name	Aarya Suhas Kudale
PRN Number	1272250489
Course	Master of Computer Applications (MCA)
Subject	Computer Vision
Assignment	CCA 3
Primary Artifact	CV_Aarya.ipynb (Google Colab notebook)
Execution Environment	Google Colab
Programming Language	Python
Submission Date	November 2026

Executive Project Overview

This report documents the implementation of four computer vision areas: (1) Image Processing and Segmentation, (2) Feature Extraction and Matching, (3) Object Detection, and (4) Video Motion Estimation. The practical work was carried out in Google Colab using Python, OpenCV, scikit-image, NumPy, pandas, Matplotlib and the Ultralytics YOLO library.

The notebook uses two photographs of a dog for image processing and feature analysis, a face photograph for face detection, and a traffic video for object detection and motion analysis. Screenshot placeholders are included throughout this document. Replace each placeholder with the corresponding output screenshot from the completed Colab notebook.

Module	Implemented Techniques	Evidence to Insert
Task 1: Image Processing & Segmentation	Noise generation, median/Gaussian filtering, PSNR, SSIM, Otsu thresholding, morphology, Canny edges and Watershed.	Input photos, noise/filtering, metrics, binary/morphology, edges and Watershed outputs.
Task 2: Feature Extraction & Matching	HOG, SIFT keypoints, Brute-Force matching, Lowe's Ratio Test, rotation and resizing comparison.	HOG, SIFT keypoints, matching and comparison outputs.
Task 3: Object Detection	Viola-Jones Haar Cascade and YOLOv8.	Face detection and YOLO traffic-frame output.
Task 4: Video Motion Estimation	Frame extraction, frame differencing, GMM/MOG2 and Lucas-Kanade optical flow.	Extracted frames, difference mask, GMM masks and optical-flow output.

1. Task 1: Image Processing and Segmentation

Objective: To apply image-processing operations to the supplied dog photograph, study the effect of noise and filtering, evaluate restoration quality, and explore thresholding, morphological processing, edge detection and image segmentation.

1.1 Input Image Loading

Two dog photographs were uploaded to Google Colab and copied into the project input folder. OpenCV was used to read the images. The images were converted from BGR to RGB format for correct display using Matplotlib.

```
image1_path = "photo_original.jpg"
image2_path = "photo_second.jpg"

original = cv2.imread(input_folder + "/photo_original.jpg")
second = cv2.imread(input_folder + "/photo_second.jpg")

original_rgb = cv2.cvtColor(original, cv2.COLOR_BGR2RGB)
second_rgb = cv2.cvtColor(second, cv2.COLOR_BGR2RGB)
```



Figure 1.1: Original dog photograph and second dog photograph

Observed image dimensions recorded in the notebook:

Image	Dimensions (height, width, channels)
Original photograph	(1160, 522, 3)
Second photograph	(1334, 1179, 3)

1.2 Noise Modelling

Noise was added to the original photograph to study how image quality changes. Gaussian noise was generated using a normal distribution with mean 0 and standard deviation 25. Salt-and-pepper noise was generated by randomly replacing selected pixels with white or black values.

```
mean = 0
sigma = 25
gaussian_noise = np.random.normal(mean, sigma, original_rgb.shape)
```

```
gaussian_noisy = np.clip(
    original_rgb.astype(np.float32) + gaussian_noise, 0, 255
).astype(np.uint8)

probability = 0.04
random_values = np.random.random(original_rgb.shape[:2])
salt_pepper_noisy = original_rgb.copy()
salt_pepper_noisy[random_values < probability / 2] = 255
salt_pepper_noisy[random_values > 1 - probability / 2] = 0
```

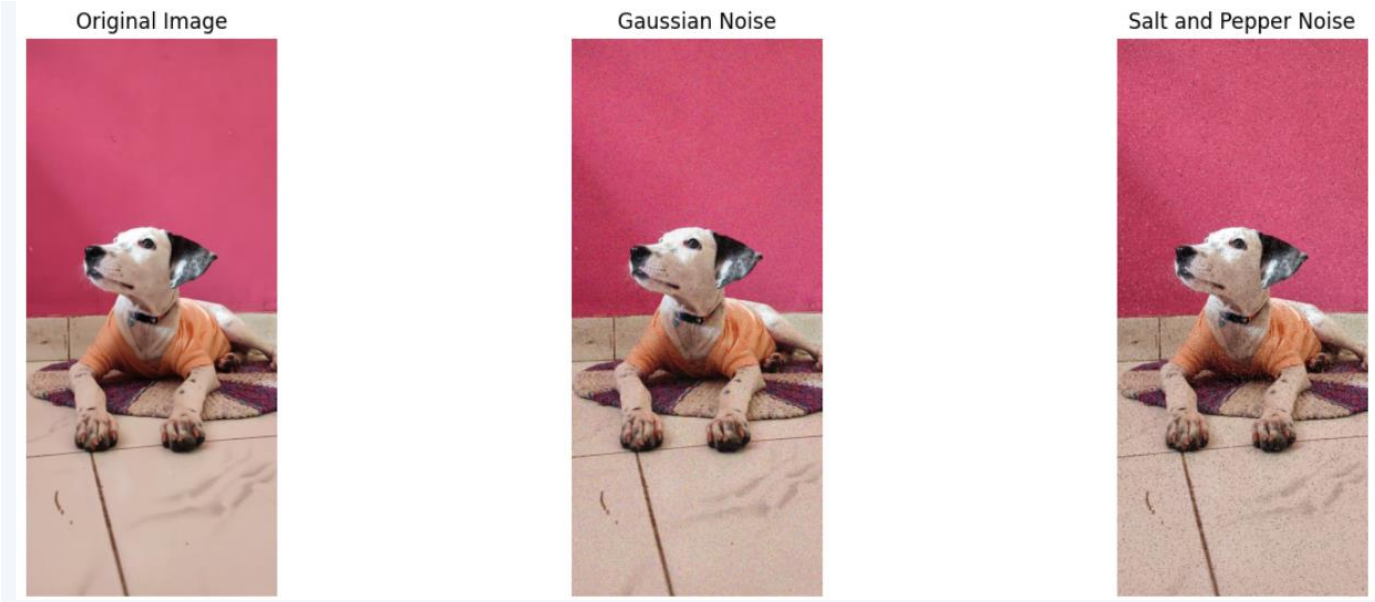


Figure 1.2: Original image, Gaussian noise and salt-and-pepper noise

1.3 Image Restoration using Spatial Filters

Median and Gaussian filters were applied to the noisy images. Median filtering replaces a pixel with the median value of its neighbourhood and is useful for reducing impulse noise. Gaussian filtering smooths an image using a weighted kernel.

```
median_gaussian = cv2.medianBlur(gaussian_noisy, 5)
median_salt_pepper = cv2.medianBlur(salt_pepper_noisy, 5)

gaussian_filtered_1 = cv2.GaussianBlur(gaussian_noisy, (5, 5), 0)
gaussian_filtered_2 = cv2.GaussianBlur(salt_pepper_noisy, (5, 5), 0)
```

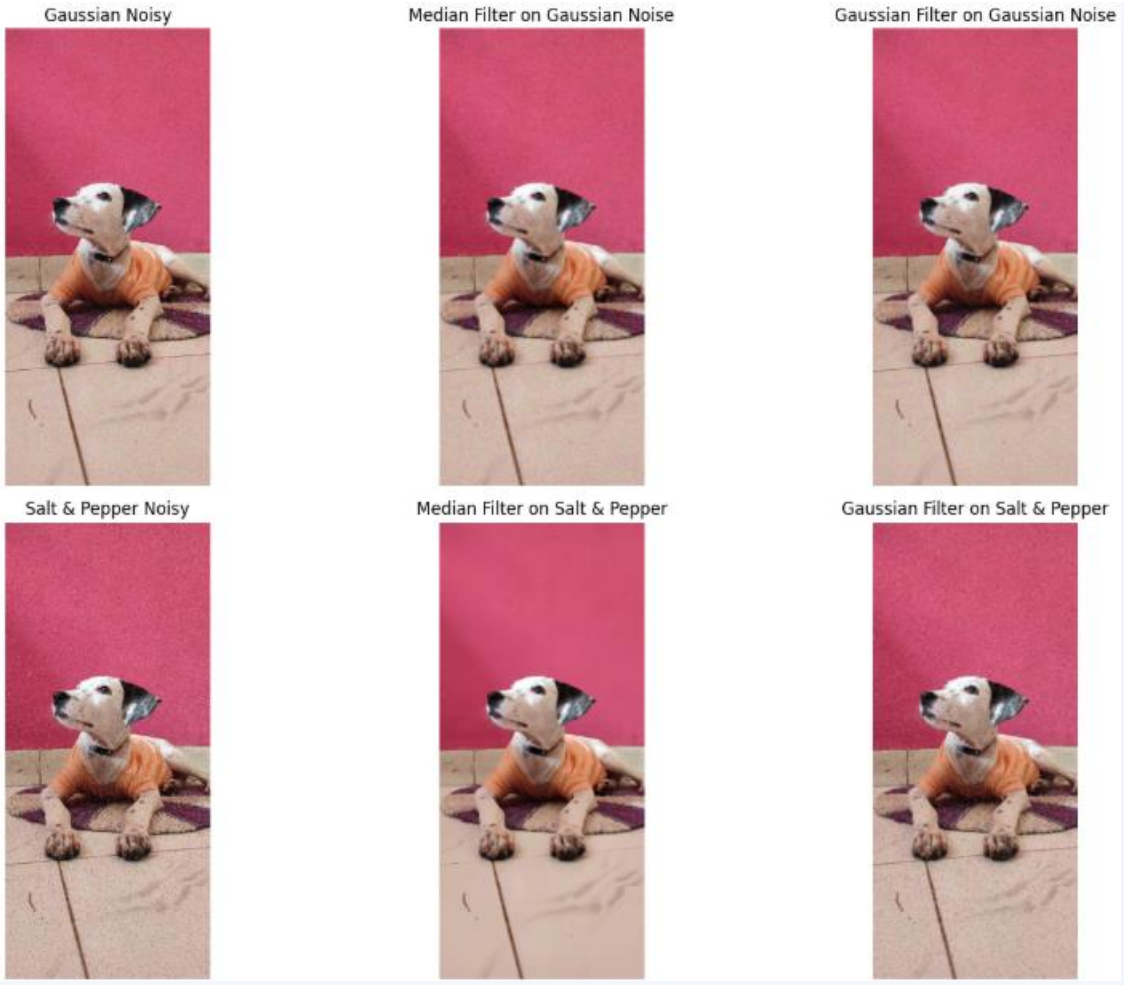


Figure 1.3: Comparison of noisy images and median/Gaussian filtered results

1. Task 1: Image Processing and Segmentation (Continued)

1.4 Image Quality Evaluation using PSNR and SSIM

Peak Signal-to-Noise Ratio (PSNR) measures reconstruction quality in decibels. Structural Similarity Index (SSIM) compares structural information, contrast and luminance between the original and processed images. Both metrics were calculated for the noisy and restored images.

```
psnr_value = peak_signal_noise_ratio(
    original_rgb, img, data_range=255
)
ssim_value = structural_similarity(
    original_rgb, img, channel_axis=2, data_range=255
)
```

Technique	PSNR (dB)	SSIM
Gaussian Noise	20.29	0.1942
Median on Gaussian	28.78	0.6899
Gaussian Filter on Gaussian	29.49	0.6677
Salt & Pepper Noise	19.20	0.3252
Median on Salt & Pepper	31.70	0.9420
Gaussian Filter on Salt & Pepper	28.57	0.6594

Observation: In the recorded results, Median on Salt & Pepper produced the highest PSNR (31.70 dB) and SSIM (0.9420) among the listed techniques.

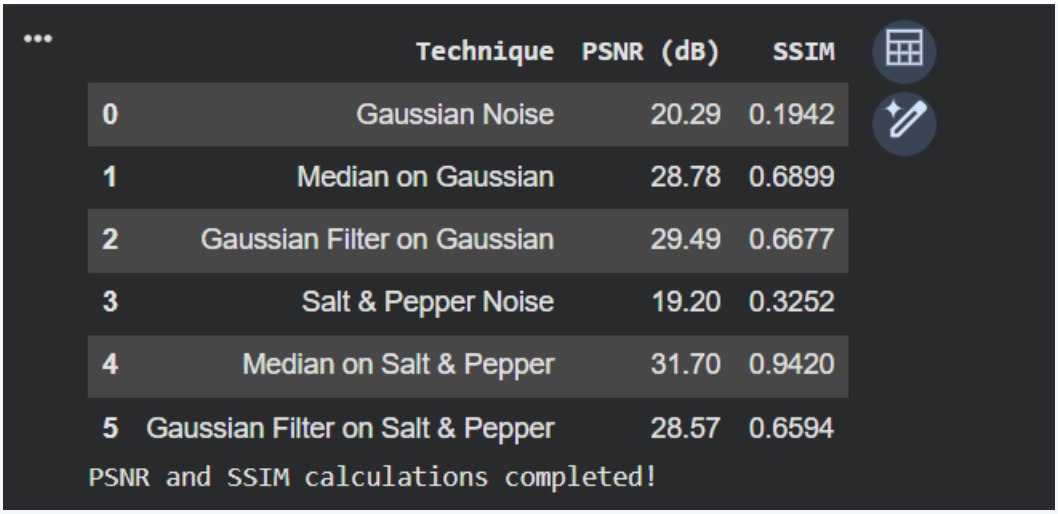


Figure 1.4: PSNR and SSIM comparison charts generated in Colab

1.5 Binary Image using Otsu Thresholding

The original image was converted to grayscale and Otsu's thresholding method was applied. Otsu's method automatically selects a threshold value to separate pixels into foreground and background.

```
gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)
threshold_value, binary_image = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Observed threshold value: 147.0



Figure 1.5: Grayscale image and binary image using Otsu thresholding

1.6 Morphological Processing

Morphological operations were applied to the binary image using a 5 × 5 kernel. Erosion reduces foreground regions, dilation expands them, opening performs erosion followed by dilation, and closing performs dilation followed by erosion.

```
kernel = np.ones((5, 5), np.uint8)
erosion = cv2.erode(binary_image, kernel, iterations=1)
dilation = cv2.dilate(binary_image, kernel, iterations=1)
opening = cv2.morphologyEx(binary_image, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(binary_image, cv2.MORPH_CLOSE, kernel)
```

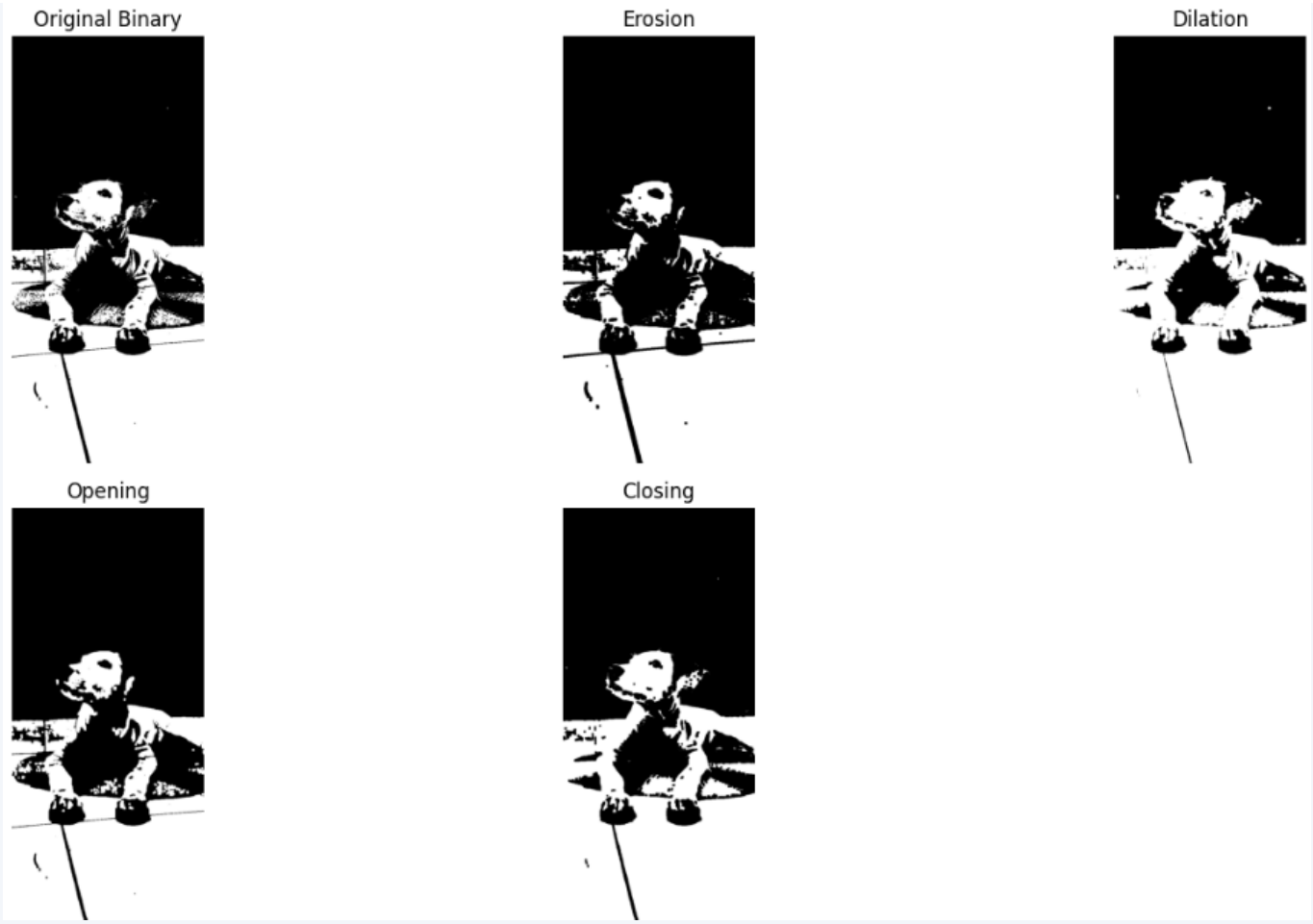


Figure 1.6: Original binary image, erosion, dilation, opening and closing

1. Task 1: Image Processing and Segmentation (Continued)

1.7 Canny Edge Detection

Canny edge detection was applied to the grayscale image to identify prominent intensity boundaries. The operation used lower and upper thresholds of 100 and 200 respectively.

```
edges = cv2.Canny(gray, 100, 200)
```

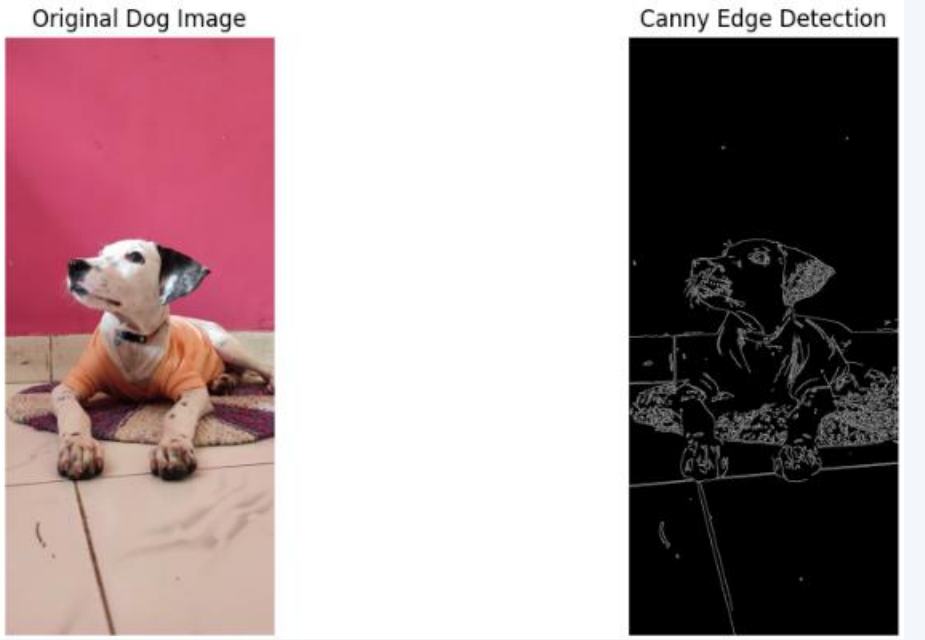


Figure 1.7: Original dog image and Canny edge detection output

1.8 Improved Watershed Segmentation

Watershed segmentation was used to identify image regions and highlight their boundaries. In this notebook, GrabCut was first used with a rectangular region around the dog to obtain a foreground estimate. Morphological opening and closing were applied to reduce noise. A sure background was created by dilation, while a distance transform and threshold were used to estimate the sure foreground. Connected components generated markers, and the Watershed algorithm refined the region boundaries.

```
# Extract foreground from the GrabCut mask
foreground = np.where(
    (mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD),
    255, 0
).astype(np.uint8)


# Create markers and apply Watershed
num_labels, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
markers = cv2.watershed(img, markers)


result = original.copy()
result[markers == -1] = [0, 0, 255]
```

The notebook output reported that improved Watershed segmentation completed and detected 1 marker region. The screenshot should show the original dog image, improved foreground mask and Watershed boundary result.

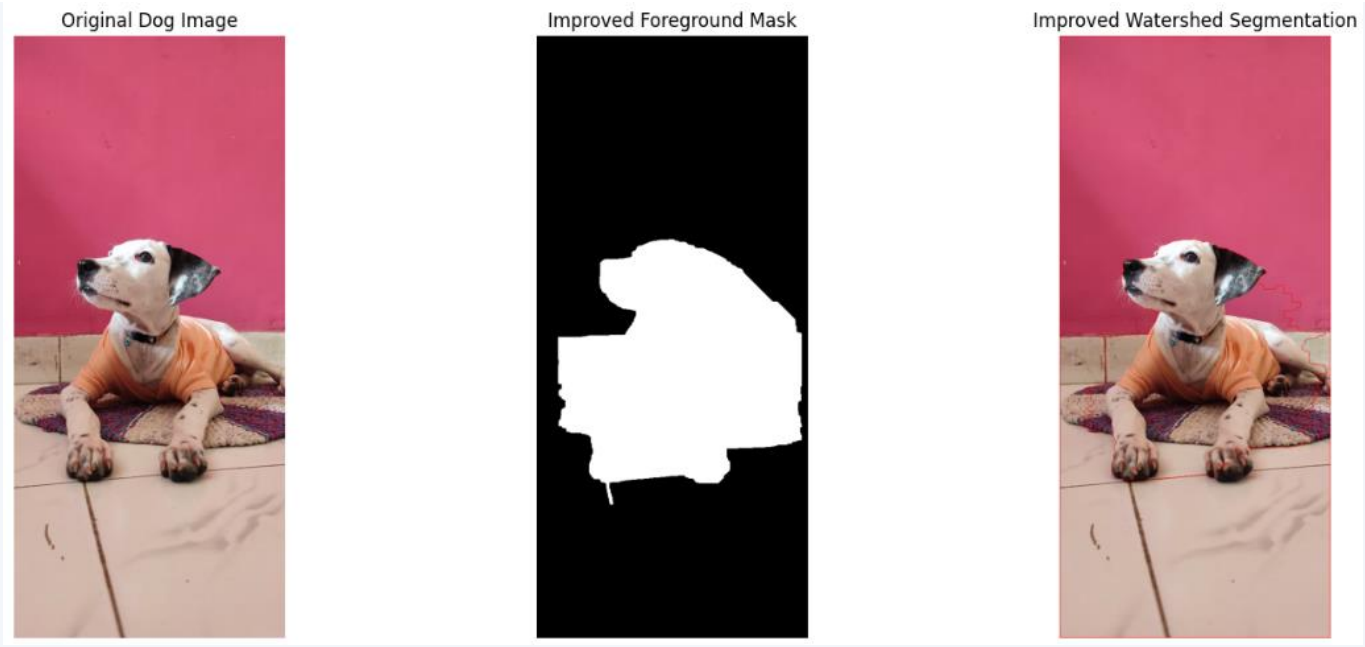


Figure 1.8: Improved Watershed segmentation – original image, foreground mask and result

An enhanced HOG visualization was also generated by rescaling its intensity for better contrast.

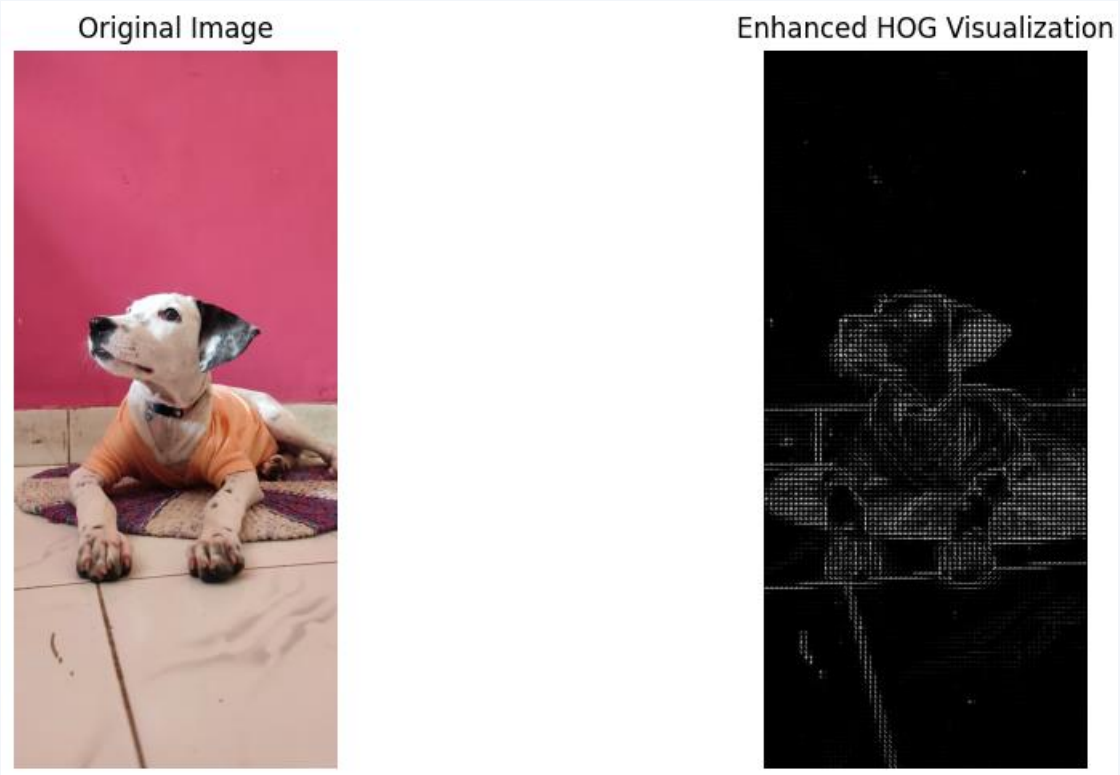


Figure 2.2: Enhanced HOG visualization

2. Task 2: Feature Extraction and Matching

Objective: To extract useful visual information from the dog photographs using Histogram of Oriented Gradients (HOG), detect distinctive SIFT keypoints, and match features between the two images.

2.1 Histogram of Oriented Gradients (HOG)

HOG is a feature descriptor that represents an image through the distribution of gradient directions. The image is divided into small cells, and gradient orientations are summarized into histograms. The notebook converted the original photograph to grayscale and extracted HOG features using 9 orientations, 8 × 8 pixels per cell and 2 × 2 cells per block with L2-Hys normalization.

```
hog_features, hog_image = hog(
    gray_hog,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)
```

Observed output: 331,776 total HOG features.

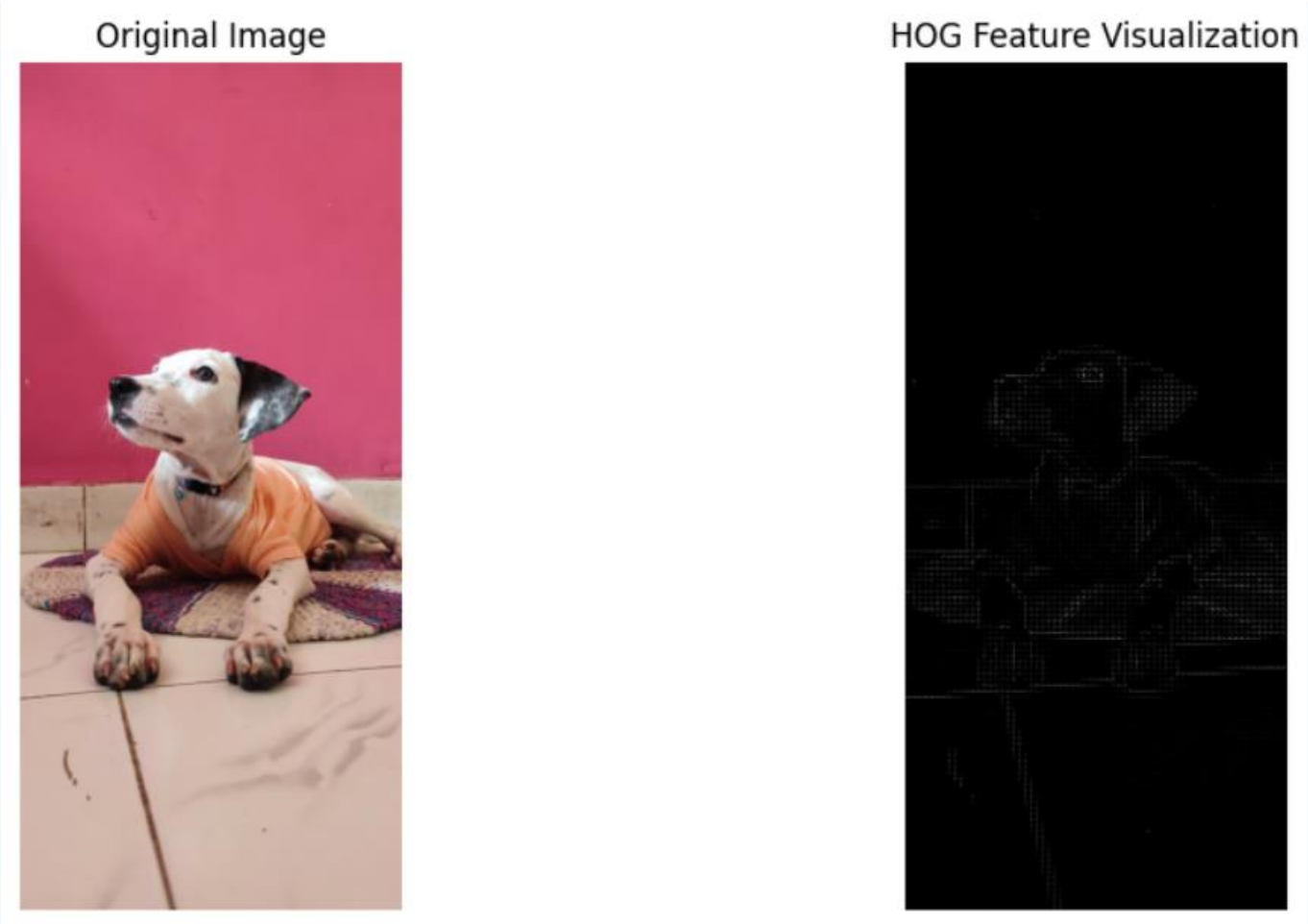


Figure 2.1: Original dog photograph and HOG feature visualization

2.2 SIFT Keypoint Detection

Scale-Invariant Feature Transform (SIFT) detects distinctive keypoints and computes a descriptor for each point. It is designed to identify local image features under changes in scale and rotation. SIFT was applied independently to the original and second dog photographs.

```
sift = cv2.SIFT_create()
keypoints1, descriptors1 = sift.detectAndCompute(original, None)
keypoints2, descriptors2 = sift.detectAndCompute(second, None)
```

Measurement	Observed Result
Keypoints in original image	1,294
Keypoints in second image	6,682
Descriptor shape for original image	(1294, 128)

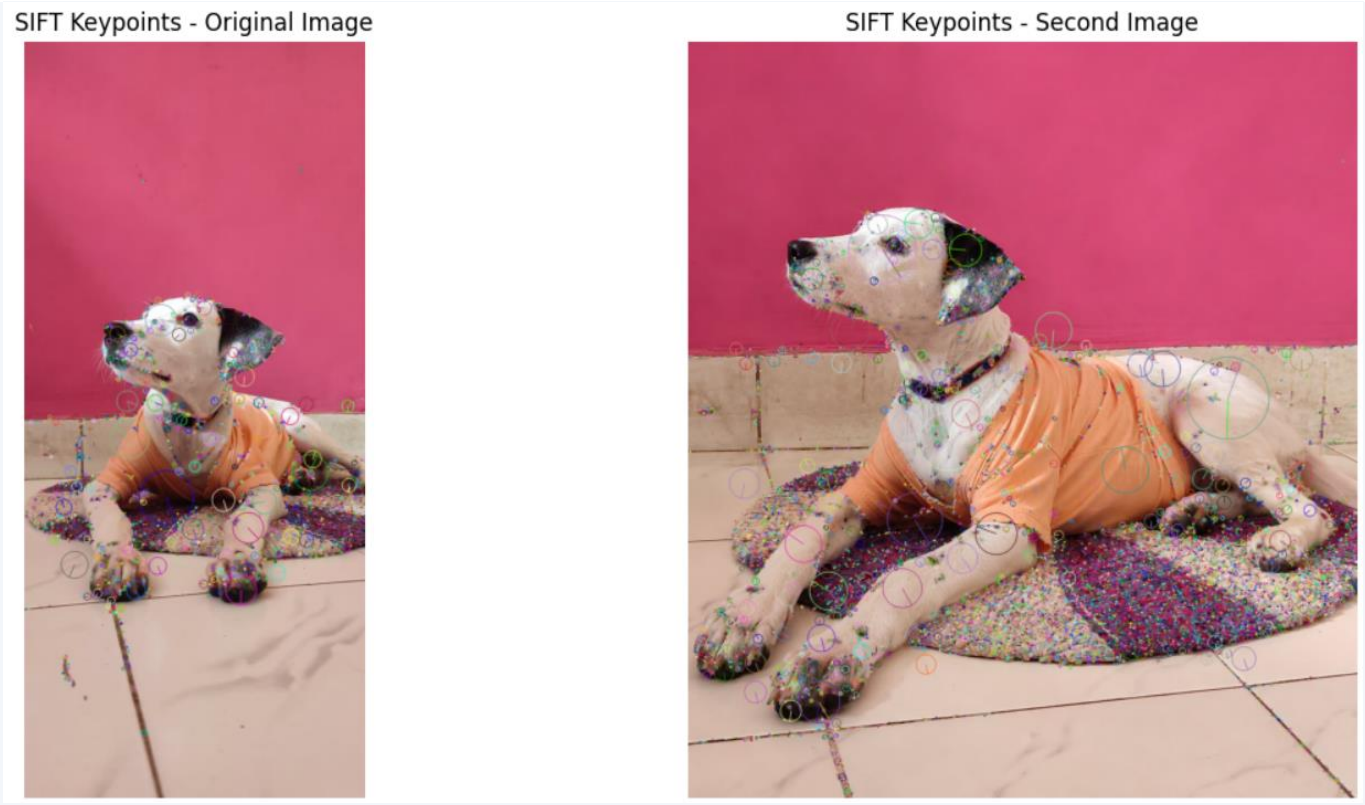


Figure 2.3: SIFT keypoints detected on original and second photographs

2. Task 2: Feature Extraction and Matching (Continued)

2.3 SIFT Feature Matching

Feature matching compares descriptors from two images to identify corresponding visual features. A Brute-Force Matcher was used to compare descriptors. Lowe's Ratio Test retained stronger matches by accepting a match when its distance was less than 0.75 times the distance of the second-nearest match.

```
bf = cv2.BFMatcher()
matches = bf.knnMatch(descriptors1, descriptors2, k=2)

good_matches = []
for pair in matches:
    if len(pair) == 2:
        m, n = pair
        if m.distance < 0.75 * n.distance:
            good_matches.append(m)
```

Observed output: 1,294 initial matches were examined and 26 good matches remained after applying Lowe's Ratio Test.



Figure 2.4: SIFT feature matching between the original and second dog photographs

2.4 Matching after Rotation

The second photograph was rotated by 45 degrees. SIFT features were detected again in the rotated image and matched against descriptors from the original photograph to examine matching under rotation.

```
rotation_matrix = cv2.getRotationMatrix2D(center, 45, 1.0)
```

```
rotated_image = cv2.warpAffine(second, rotation_matrix, (w, h))
```

Observed output: 22 good matches after rotation.



Figure 2.5: SIFT matching after 45-degree rotation

2.5 Matching after Resizing

The second photograph was resized to 70% of its original width and height. SIFT descriptors were computed for the resized image and matched with the original photograph.

```
resized_image = cv2.resize(
    second, None, fx=0.7, fy=0.7,
    interpolation=cv2.INTER_AREA
)
```

Observed output: 36 good matches after resizing.

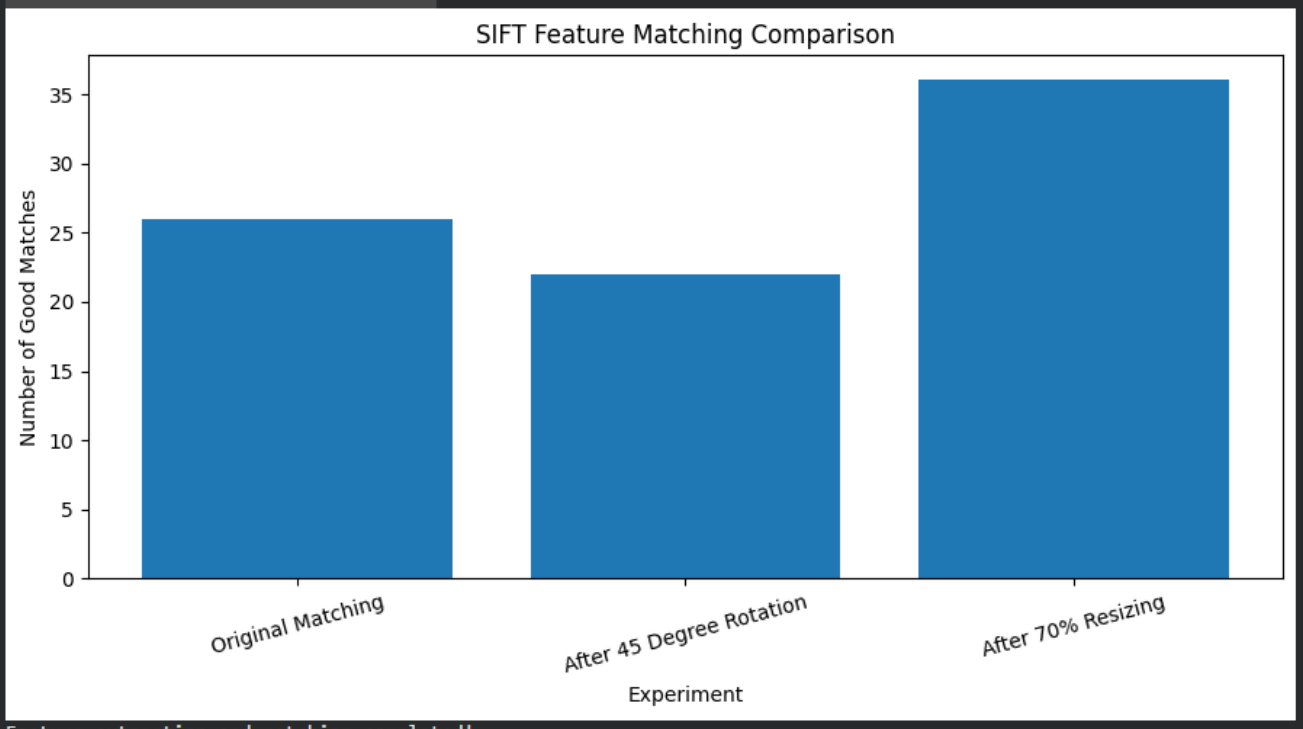


Figure 2.6: SIFT matching after 70% resizing

2.6 Feature Matching Comparison

Experiment	Good Matches
Original Matching	26
After 45 Degree Rotation	22
After 70% Resizing	36

	Experiment	Good Matches
0	Original Matching	26
1	After 45 Degree Rotation	22
2	After 70% Resizing	36




```
Feature extraction and matching completed!
```

Figure 2.7: SIFT feature matching comparison table and graph

3. Task 3: Object Detection

Objective: To implement and compare a classical face-detection approach (Viola-Jones Haar Cascade) with a deep-learning object-detection approach (YOLOv8).

3.1 Viola-Jones Face Detection

Viola-Jones is a classical object-detection method commonly used for frontal face detection. The Haar Cascade classifier evaluates image regions using Haar-like features and a cascade of classifiers. The uploaded face photograph was converted to grayscale, processed by the pretrained frontal-face cascade, and a rectangle was drawn around each detected face.

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)
faces = face_cascade.detectMultiScale(
    gray_face, scaleFactor=1.1,
    minNeighbors=5, minSize=(30, 30)
)
```

Observed output: 1 face detected.

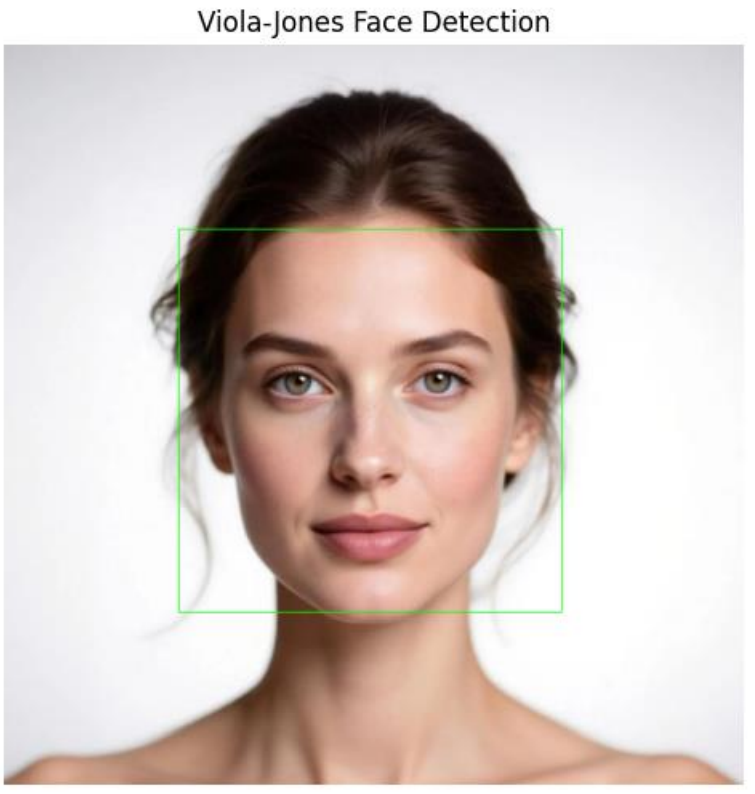


Figure 3.1: Viola-Jones face detection output with bounding box

3.2 YOLOv8 Object Detection

YOLO (You Only Look Once) is a deep-learning object detector. It predicts object classes, bounding boxes and confidence scores. The pretrained YOLOv8n model was applied to a sample frame extracted from the traffic video. The annotated frame was displayed with the detected labels.

```
model = YOLO("yolov8n.pt")
results = model.predict(
    source=sample_frame,
    conf=0.25,
    verbose=False
)
yolo_result = results[0].plot()
```

The notebook printed six detections, all classified as cars.

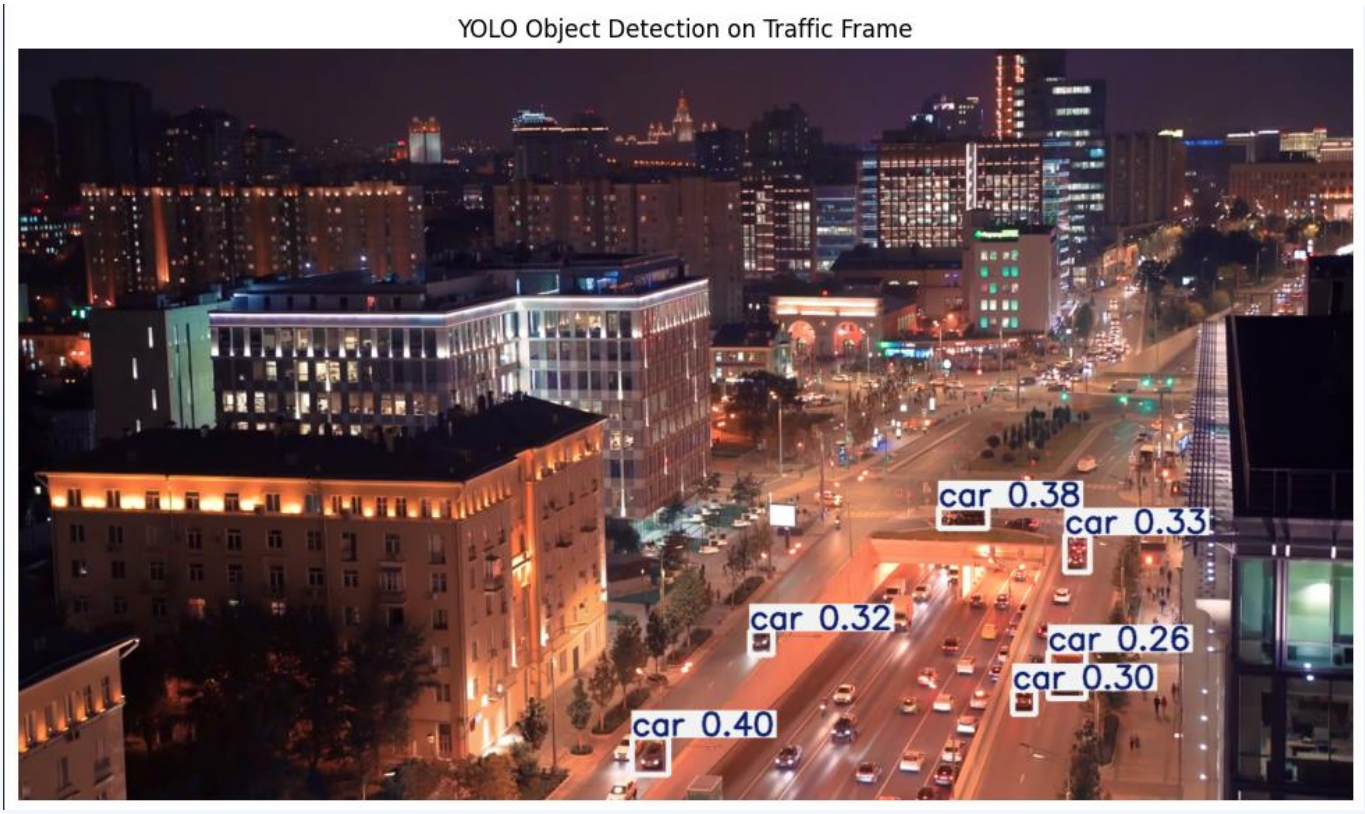


Figure 3.2: YOLOv8 object detection on the traffic-video frame

3. Task 3: Object Detection (Continued)

3.3 YOLOv8 Detection Output Analysis

Detected Class	Confidence Score
Car	0.399
Car	0.383
Car	0.334
Car	0.317
Car	0.303
Car	0.257

These confidence scores represent the model's confidence for the corresponding detected bounding boxes. They are not overall model accuracy percentages.

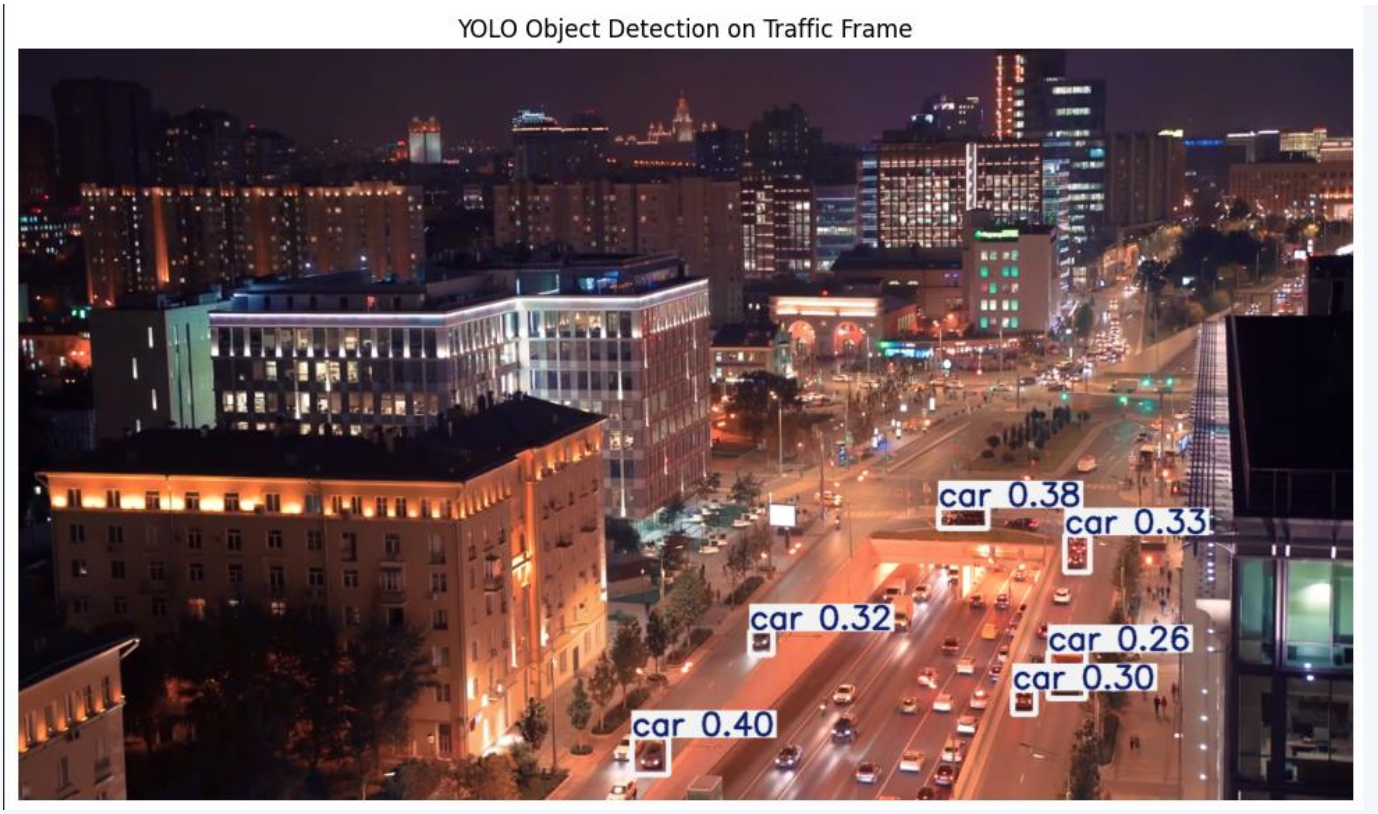


Figure 3.3: YOLOv8 detection output and printed confidence scores

3.4 Comparison of Detection Methods

Parameter	Viola-Jones Haar Cascade	YOLOv8
Main purpose	Frontal human face detection	General object detection
Detection result	1 face detected	6 car detections in the displayed frame
Approach	Classical cascade classifier	Deep-learning object detector
Output	Face bounding box	Class labels, bounding boxes and confidence scores

3.5 Result

Viola-Jones successfully detected one face in the uploaded photograph. YOLOv8 detected six cars in the selected traffic-video frame and displayed confidence scores for the detections.

4. Task 4: Video Motion Estimation

Objective: To analyze motion in a traffic video by inspecting the video stream, extracting frames, and applying frame differencing, Lucas-Kanade optical flow and GMM/MOG2 background subtraction.

4.1 Video Stream Inspection

The traffic video was loaded using OpenCV VideoCapture. The notebook read the video's frame rate, total frame count, frame dimensions and duration. A sample frame was also extracted for visual inspection.

Property	Observed Value
Frame rate	25.0 FPS
Total frames	782
Resolution	1280 × 720
Duration	31.28 seconds

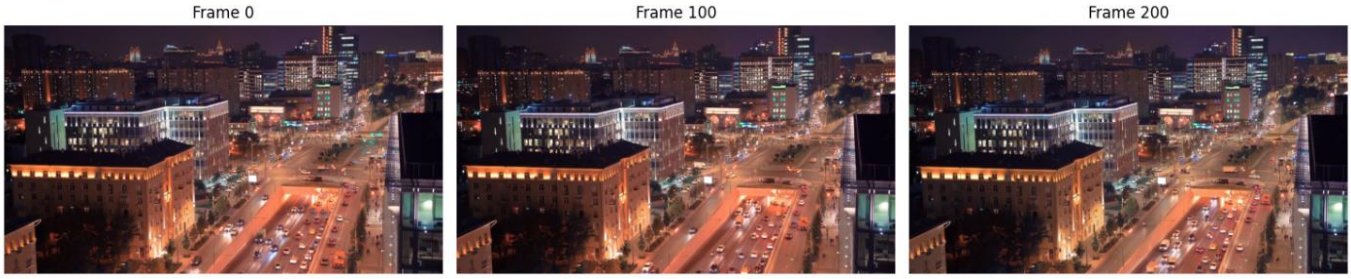


Figure 4.1: Sample frame from the traffic video

4.2 Video Frame Extraction

Frames were read sequentially from the traffic video. Each frame was resized to 640 × 360 to make subsequent processing faster. Selected frames (0, 100 and 200) were displayed to verify extraction.

```
cap = cv2.VideoCapture(video_path)
frames = []

while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 360))
    frames.append(frame)

cap.release()
```

Observed output: 780 frames extracted.

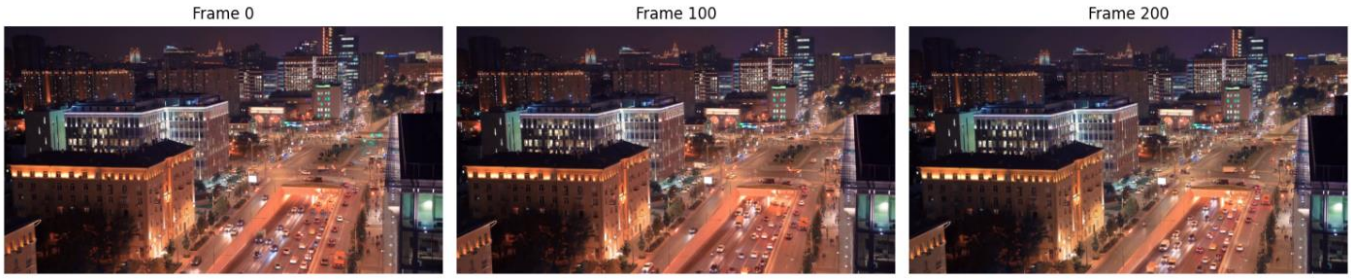


Figure 4.2: Extracted video frames – Frame 0, Frame 100 and Frame 200

4. Task 4: Video Motion Estimation (Continued)

4.3 Method 1: Frame Differencing

Frame differencing detects changes between consecutive frames. The selected frames were converted to grayscale, and the absolute pixel-wise difference was calculated. Thresholding produced a binary mask highlighting regions with sufficient change.

```
difference = cv2.absdiff(gray_frames[100], gray_frames[101])
_, foreground_mask = cv2.threshold(
    difference, 25, 255, cv2.THRESH_BINARY
)
```



Figure 4.3: First frame, second frame and frame-differencing mask

4.4 Method 2: Lucas-Kanade Optical Flow

Lucas-Kanade optical flow estimates movement of selected feature points between consecutive frames. The notebook detected trackable corners in one frame and used calcOpticalFlowPyrLK() to estimate their positions in the next frame. Motion arrows were drawn to visualize point movement.

```
p0 = cv2.goodFeaturesToTrack(
    old_gray, mask=None, **feature_params
)
p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray, new_gray, p0, None, **lk_params
)
```

Observed output: 100 initial feature points and 100 successfully tracked points.

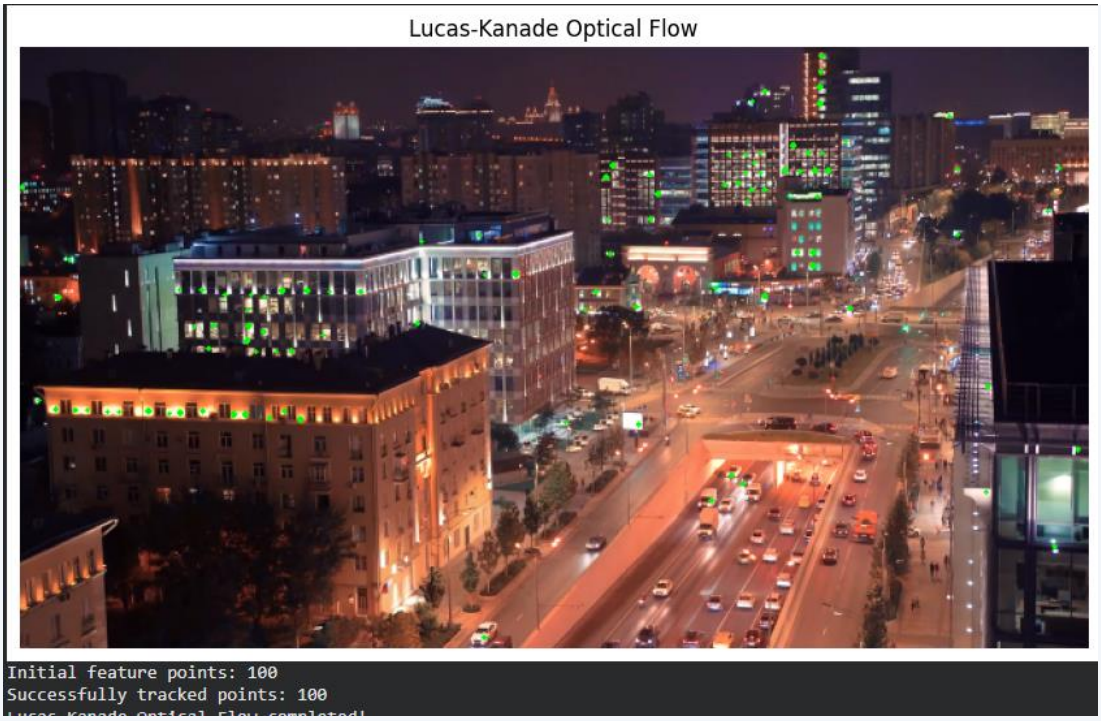


Figure 4.4: Lucas-Kanade optical flow with tracked feature points

4. Task 4: Video Motion Estimation (Continued)

4.5 Method 3: GMM Background Subtraction

Gaussian Mixture Model (GMM) background subtraction models the background at each pixel using a mixture of Gaussian distributions. OpenCV's MOG2 implementation updates the background model as frames are processed and generates a foreground mask.

```
background_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=25,
    detectShadows=True
)
foreground_mask = background_subtractor.apply(frame)
```

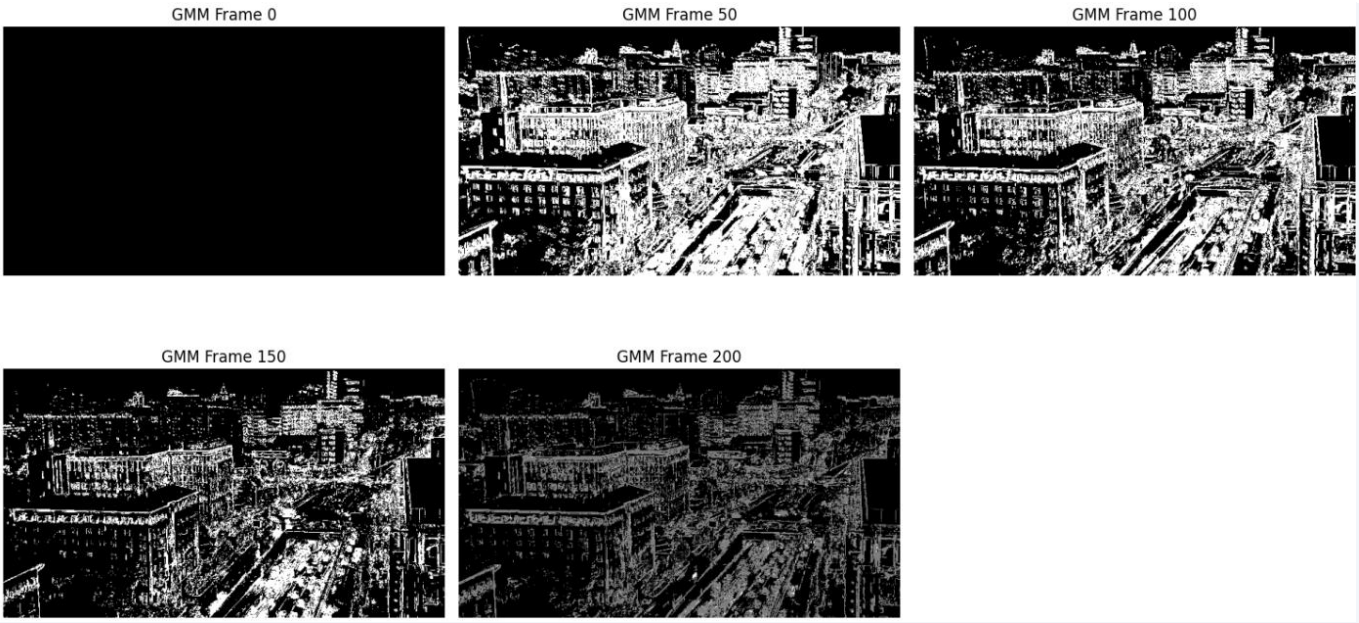


Figure 4.5: GMM/MOG2 foreground masks for selected video frames

4.6 Motion Estimation Summary

Technique	Purpose	Observed Output
Frame Extraction	Read video frames for analysis	780 frames extracted
Frame Differencing	Highlight pixel changes between frames	Binary difference mask generated
Lucas-Kanade Optical Flow	Track feature-point movement	100 initial points; 100 successfully tracked
GMM / MOG2	Separate moving foreground from background	Foreground masks generated for sampled frames

5. Overall Experimental Results

The following table summarizes the numerical results recorded in the completed Google Colab notebook.

Experiment	Observed Result
Original and second dog photographs	Loaded successfully; dimensions (1160, 522, 3) and (1334, 1179, 3)
Image restoration – best listed PSNR	31.70 dB (Median on Salt & Pepper)
Image restoration – best listed SSIM	0.9420 (Median on Salt & Pepper)
Otsu thresholding	Threshold value: 147.0
Improved Watershed segmentation	Completed; 1 marker region reported
HOG feature extraction	331,776 features
SIFT keypoint detection	1,294 keypoints in original; 6,682 in second image
SIFT original matching	26 good matches
SIFT after 45° rotation	22 good matches
SIFT after 70% resizing	36 good matches
Viola-Jones face detection	1 face detected
YOLOv8 object detection	6 car detections
Traffic video inspection	25 FPS; 782 frames; 1280 × 720; 31.28 seconds
Video frame extraction	780 frames extracted
Lucas-Kanade optical flow	100 initial points; 100 successfully tracked
Frame differencing / GMM	Difference mask and foreground masks generated

6. Key Observations

- Gaussian and salt-and-pepper noise were added to the original dog photograph, and median and Gaussian filters were compared.
- The recorded PSNR and SSIM values show that median filtering performed best for the salt-and-pepper noisy image among the tested methods.
- Otsu thresholding, morphological operations and Canny edge detection were applied to explore image segmentation and edge information.
- The improved Watershed pipeline used a GrabCut foreground estimate before marker generation and Watershed processing.
- HOG generated 331,776 features, while SIFT detected keypoints in both dog photographs.
- SIFT matching produced 26 good matches for the original pair, 22 after rotation and 36 after resizing.
- Viola-Jones detected one face, while YOLOv8 identified six cars in the selected traffic frame.
- The traffic video was processed to extract frames, identify frame differences, generate GMM/MOG2 masks and track optical-flow points.

7. Conclusion

This CCA 3 assignment demonstrated the implementation of several computer vision techniques using Python and Google Colab. The practical work covered image processing and segmentation, feature extraction and matching, object detection, and video motion estimation.

The image-processing experiments demonstrated noise generation, image restoration, quality evaluation, thresholding, morphological operations, edge detection and Watershed segmentation. The feature-analysis experiments used HOG and SIFT to extract and match visual features, including after rotation and resizing.

The object-detection experiments demonstrated face detection using a Haar Cascade classifier and vehicle detection using YOLOv8. The video-processing experiments demonstrated frame extraction, frame differencing, Lucas-Kanade optical flow and GMM/MOG2 background subtraction. The outputs generated in Google Colab provide visual evidence of the completed implementation.

Submission Details

Field	Details
Candidate Name	Aarya Suhas Kudale
PRN Number	1272250489
Course Module	Computer Vision – CCA 3
Primary Notebook	CV_Aarya.ipynb
Execution Platform	Google Colab
Programming Language	Python